

Subject

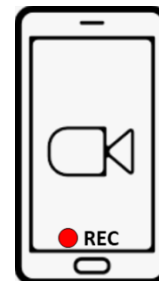
Speech and Natural Language Processing

Professor

Isaac González



Aula Virtual U/E



Participar



Session 11

Unit III

Transformers and Large Language Models

Topic 3.3

Masked language Models

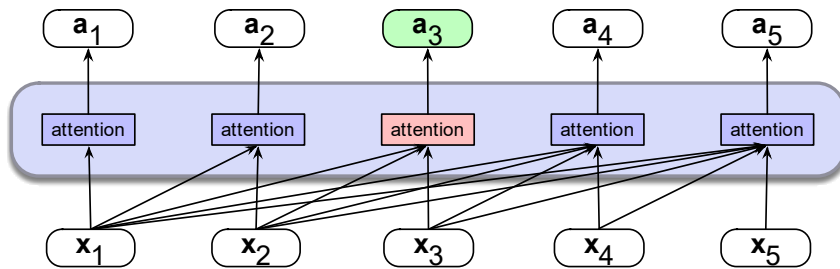
Masked Language Models

BERT

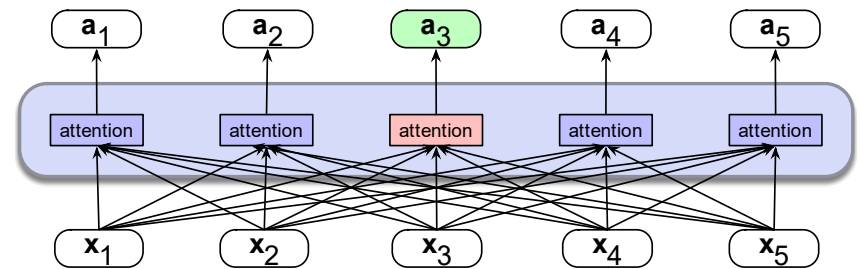
Masked Language Modeling

- We've seen autoregressive (causal, left-to-right) LMs.
- But what about tasks for which we want to peak at future tokens?
 - Especially true for tasks where we map each input token to an output token
- **Bidirectional** encoders use **masked self-attention** to
 - map sequences of input embeddings $(x_1, \dots, x_n)$
 - to sequences of output embeddings of the same length $(h_1, \dots, h_n)$,
 - where the output vectors have been contextualized using information from the entire input sequence.

Bidirectional Self-Attention



a) A causal self-attention layer



b) A bidirectional self-attention layer

Easy! We just remove the mask

Casual self-attention

N	q1•k1	$-\infty$	$-\infty$	$-\infty$
	q2•k1	q2•k2	$-\infty$	$-\infty$
	q3•k1	q3•k2	q3•k3	$-\infty$
	q4•k1	q4•k2	q4•k3	q4•k4

N

$$\mathbf{head} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

Bidirectional self-attention

N	q1•k1	q1•k2	q1•k3	q1•k4
	q2•k1	q2•k2	q2•k3	q2•k4
	q3•k1	q3•k2	q3•k3	q3•k4
	q4•k1	q4•k2	q4•k3	q4•k4

N

$$\mathbf{head} = \text{softmax} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \mathbf{V}$$

BERT: Bidirectional Encoder Representations from Transformers

BERT (Devlin et al., 2019)

- 30,000 English-only tokens (WordPiece tokenizer)
- Input context window $N=512$ tokens, and model dimensionality $d=768$
- $L=12$ layers of transformer blocks, each with $A=12$ (bidirectional) multihead-attention layers.
- The resulting model has about 100M parameters.

XLM-RoBERTa (Conneau et al., 2020)

- 250,000 multilingual tokens (SentencePiece Unigram LM tokenizer)
- Input context window $N=512$ tokens, model dimensionality $d=1024$
- $L=24$ layers of transformer blocks, with $A=16$ multihead attention layers each
- The resulting model has about 550M parameters.

Masked Language Models

BERT

Masked Language Models

Masked LM training

Masked training intuition

- For **left-to-right LMs**, the model tries to predict the last word from prior words:

The water of Walden Pond is so beautifully_____

- And we train it to improve its predictions.
- For **bidirectional masked LMs**, the model tries to predict one or more words from all the rest of the words:

The_____ of Walden Pond_____ so beautifully blue

- The model generates a probability distribution over the vocabulary for each missing token
- We use the cross-entropy loss from each of the model's predictions to drive the learning process.

MLM training in BERT

15% of the tokens are randomly chosen to be part of the masking

Example: "Lunch was **delicious**", if delicious was randomly chosen:

Three possibilities:

1. 80%: Token is replaced with special token [MASK]

Lunch was **delicious** -> Lunch was **[MASK]**

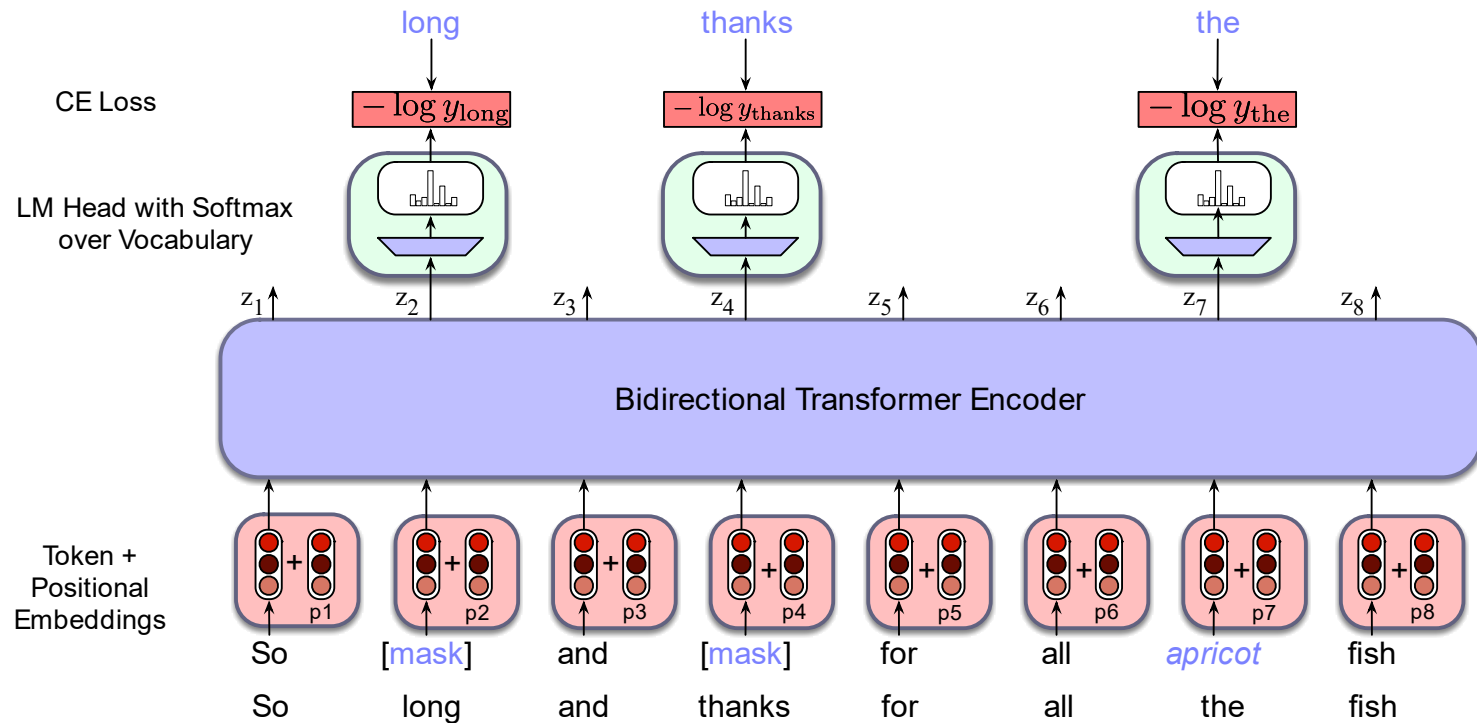
2. 10%: Token is replaced with a random token (sampled from unigram prob)

Lunch was **delicious** -> Lunch was **gasp**

3. 10%: Token is unchanged

Lunch was **delicious** -> Lunch was **delicious**

In detail



MLM loss

The LM head takes output of final transformer layer L , multiplies it by unembedding layer and turns into probabilities:

$$\mathbf{u}_i = \mathbf{h}_i^L \mathbf{E}^T$$

$$\mathbf{y}_i = \text{softmax}(\mathbf{u}_i)$$

E.g., for the x_i corresponding to "long", the loss is the probability of the correct word *long*, given output $\mathbf{h}_i^L$):

$$L_{MLM}(x_i) = -\log P(x_i | \mathbf{h}_i^L)$$

We get the gradients by taking the average of this loss over the batch

$$L_{MLM} = -\frac{1}{|M|} \sum_{i \in M} \log P(x_i | \mathbf{h}_i^L)$$

Next Sentence Prediction

Given 2 sentences the model predicts if they are a real pair of adjacent sentences from the training corpus or a pair of unrelated sentences.

BERT introduces two special tokens

- [CLS] is prepended to the input sentence pair,
- [SEP] is placed between the sentences, and also after second sentence

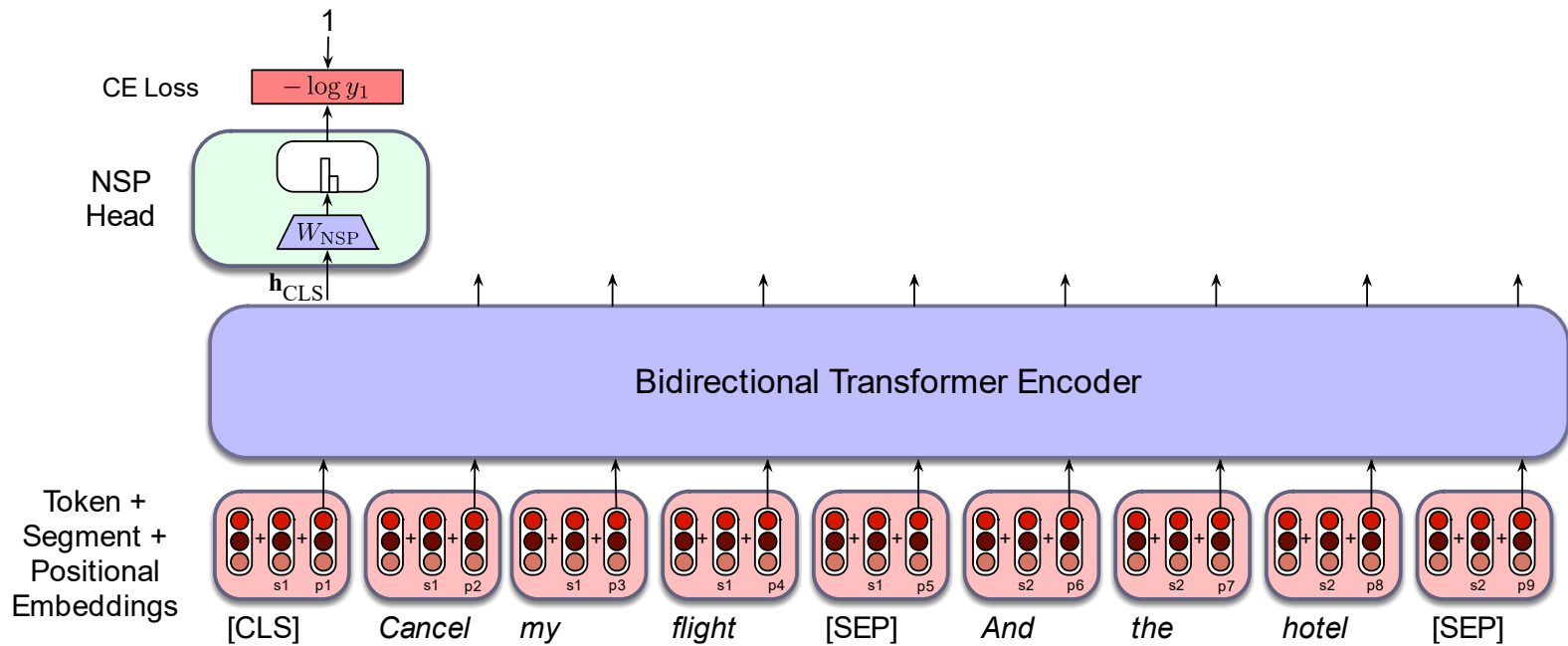
And two more special tokens

- [1st segment] and [2nd segment]
- These are added to the input embedding and positional embedding

h_{CLS}^L from the final layer [CLS] token is input to classifier head (weights W_{NSP}) that predicts two classes:.

$$y_i = \text{softmax}(h_{\text{CLS}}^L W_{\text{NSP}})$$

NSP Loss with classification head



More details

Original model was trained with 40 passes over training data

Some models (like RoBERTa) drop NSP loss

Tokenizer for multilingual models is trained from stratified sample of languages (some data from each language)

Multilingual models are better than monolingual models with small numbers of languages

- With large numbers of languages, monolingual models in that language can be better
- The "curse of multilinguality"

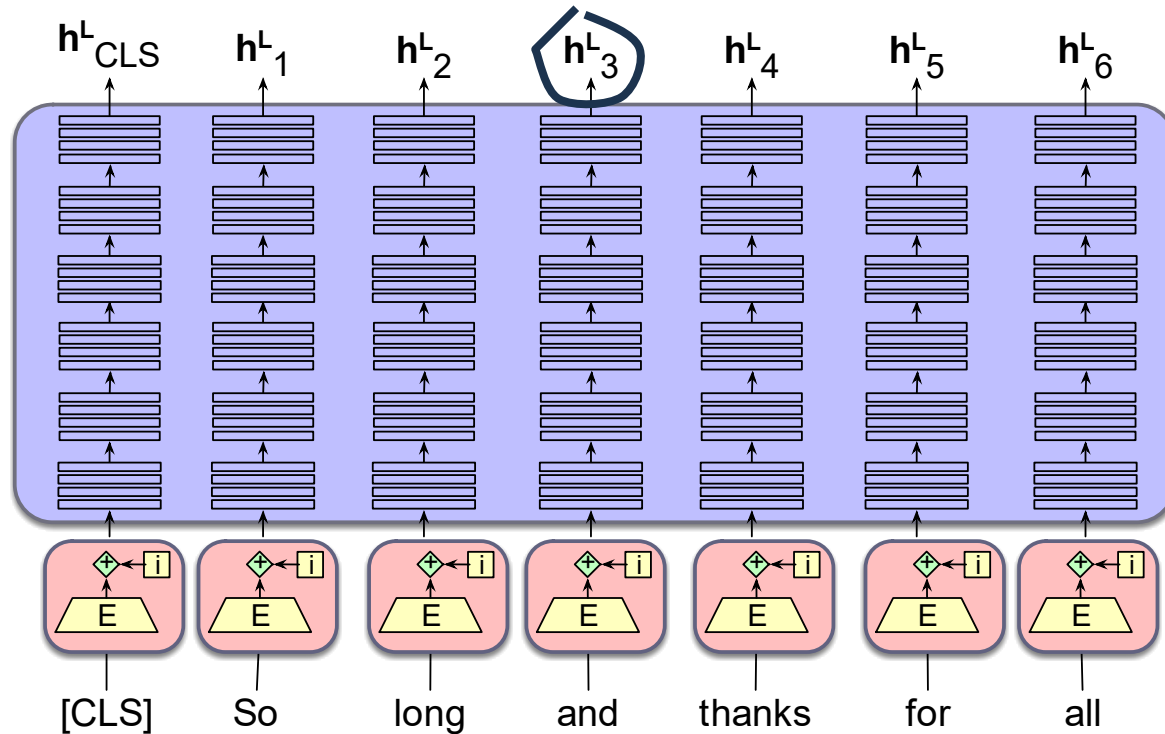
Masked Language Models

Masked LM training

Masked
Language
Models

Contextual Embeddings

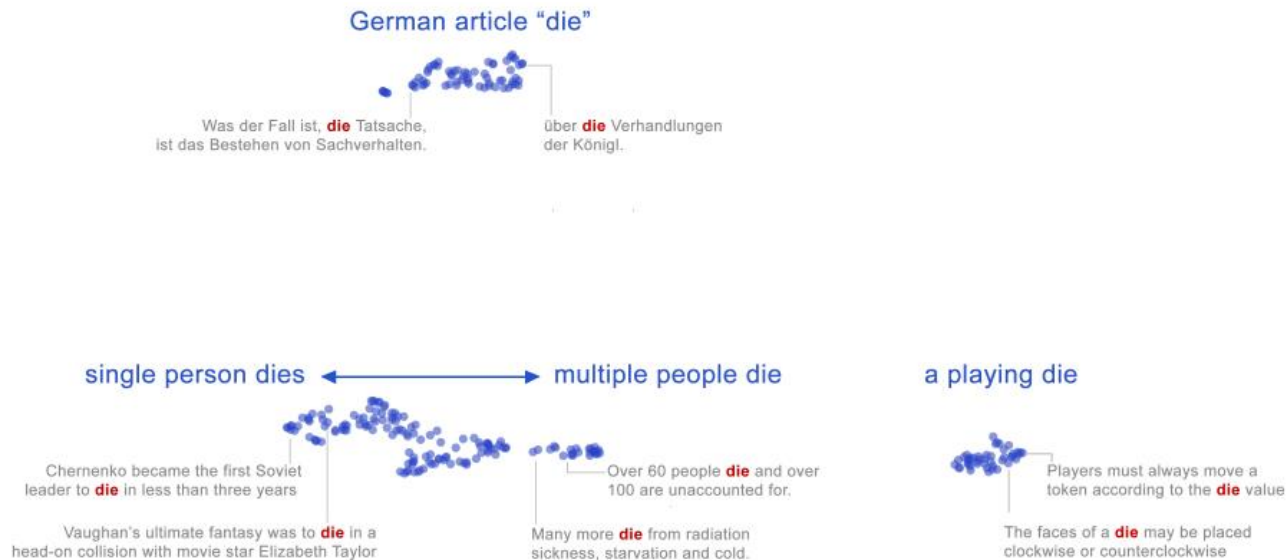
Contextual Embeddings to represent words



Static vs Contextual Embeddings

Static embeddings represent **word types** (dictionary entries)

Contextual embeddings represent **word instances** (one for each time the word occurs in any context/sentence)



Word sense

Words are ambiguous

A **word sense** is a discrete representation of one aspect of meaning

mouse¹ : a *mouse* controlling a computer system in 1968.

mouse² : a quiet animal like a *mouse*

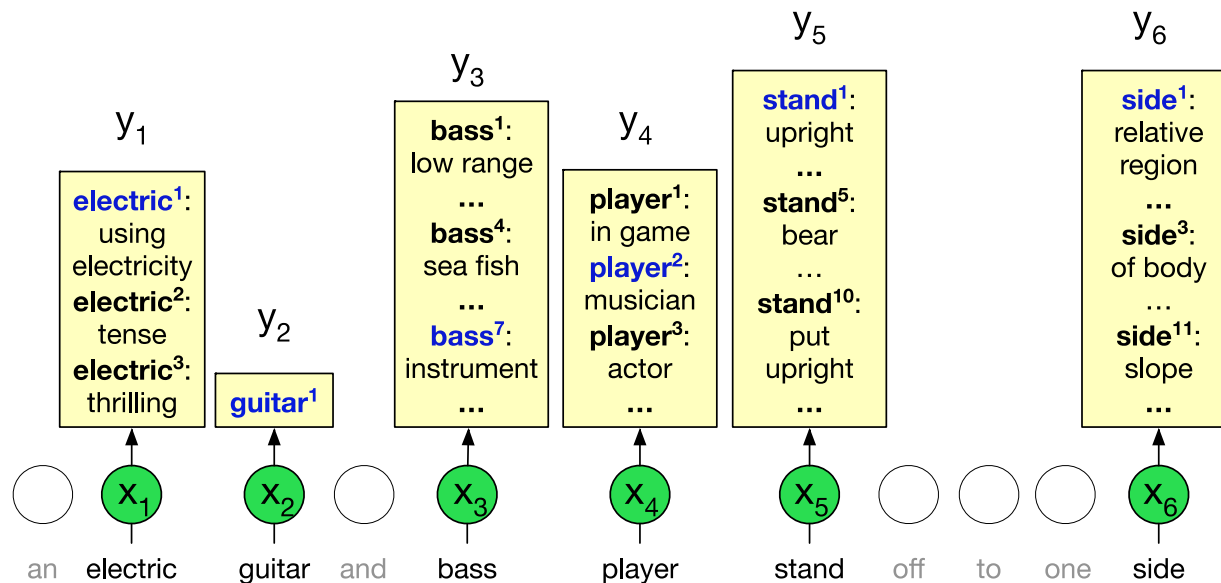
bank¹ : ...a *bank* can hold the investments in a custodial account ...

bank² : ...as agriculture burgeons on the east *bank*, the river ...

Contextual embeddings offer a continuous high-dimensional model of meaning that is more fine grained than discrete senses.

Word sense disambiguation (WSD)

The task of selecting the correct sense for a word.



1-nearest neighbor algorithm for WSD

Melamud et al (2016), Peters et al (2018)

At training time, take a sense-labeled corpus like SEMCOR

Run corpus through BERT to get contextual embedding for each token

- E.g., pooling representations from last 4 BERT transformer layer

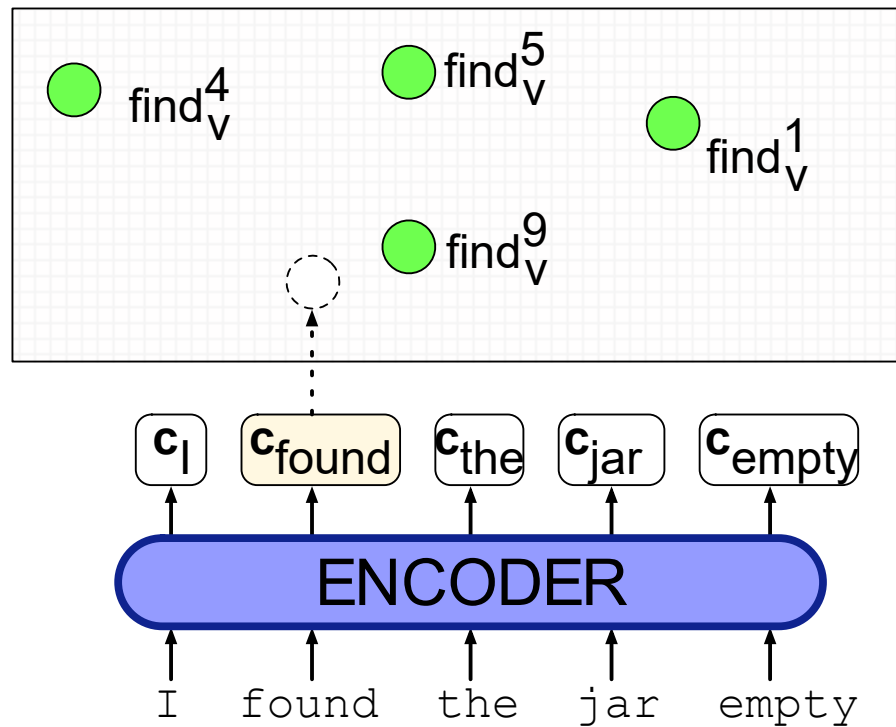
Then for each sense s of word w for n tokens of that sense, pool embeddings:

$$\mathbf{v}_s = \frac{1}{n} \sum_i \mathbf{v}_i \quad \forall \mathbf{v}_i \in \text{tokens}(s)$$

At test time, given a token of a target word t , compute contextual embedding $\mathbf{t}$ and choose its nearest neighbor sense from training set

$$\text{sense}(t) = \underset{s \in \text{senses}(t)}{\operatorname{argmax}} \cos(\mathbf{t}, \mathbf{v}_s)$$

1-nearest neighbor algorithm for WSD



Similarity and contextual embeddings

- We generally use cosine as for static embeddings
- But some issues:
 - Contextual embeddings tend to be **anisotropic**: all point in roughly the same direction so have high inherent cosines (Ethayarajh 2019)
 - Cosine measure are dominated by a small number of "rogue" dimensions with very high values (Timkey and van Schijndel 2021)
 - Cosine tends to underestimate human judgments on similarity of word meaning for very frequent words (Zhou et al., 2022)

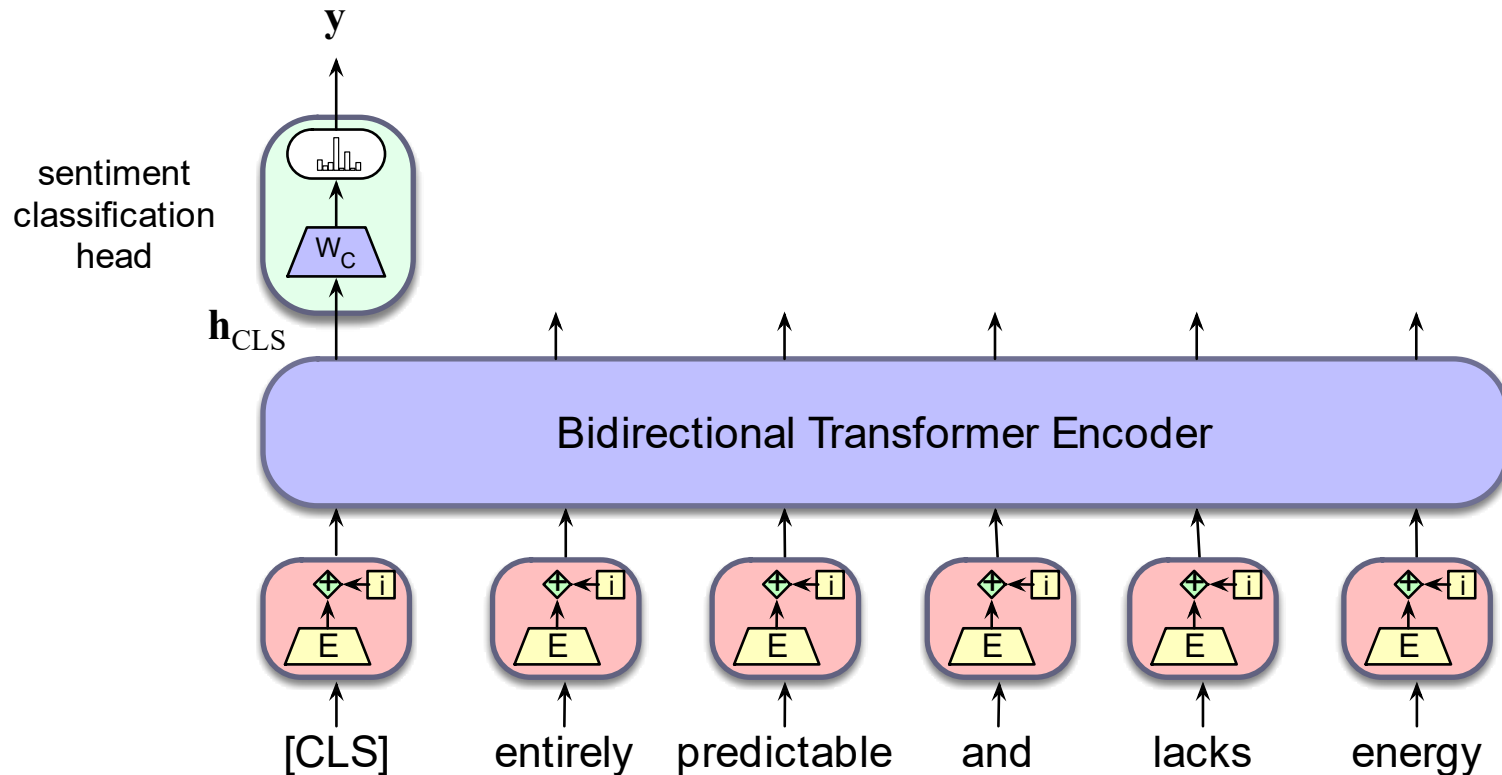
Masked
Language
Models

Contextual Embeddings

Masked
Language
Models

Fine-Tuning for Classification

Adding a sentiment classification head



Sequence-Pair classification

Assign a label to pairs of sentences:

- paraphrase detection (are the two sentences paraphrases of each other?)
- logical entailment (does sentence A logically entail sentence B?)
- discourse coherence (how coherent is sentence B as a follow-on to sentence A?)

Example: Natural Language Inference

Pairs of sentences are given one of 3 labels

- Neutral
 - a: Jon walked back to the town to the smithy.
 - b: Jon traveled back to his hometown.
- Contradicts
 - a: Tourist Information offices can be very helpful.
 - b: Tourist Information offices are never of any help.
- Entails
 - a: I'm confused.
 - b: Not all of it is very clear to me.

Algorithm: pass the premise/hypothesis pairs through a bidirectional encoder and use the output vector for the [CLS] token as the input to the classification head .

Fine-tuning for sequence labeling

Assign a label from a small fixed set of labels to each token in the sequence.

- Named entity recognition
- Part of speech tagging
-

Named Entity Recognition

A **named entity** is anything that can be referred to with a proper name: a person, a location, an organization

Named entity recognition (NER): find spans of text that constitute proper names and tag the type of the entity

Type	Tag	Sample Categories	Example sentences
People	PER	people, characters	Turing is a giant of computer science.
Organization	ORG	companies, sports teams	The IPCC warned about the cyclone.
Location	LOC	regions, mountains, seas	Mt. Sanitas is in Sunshine Canyon .
Geo-Political Entity	GPE	countries, states	Palo Alto is raising the fees for parking.

Named Entity Recognition

Citing high fuel prices, [ORG United Airlines] said [TIME Friday] it has increased fares by [MONEY \$6] per round trip on flights to some cities also served by lower-cost carriers. [ORG American Airlines], a unit of [ORG AMR Corp.], immediately matched the move, spokesman [PER Tim Wagner] said. [ORG United], a unit of [ORG UAL Corp.], said the increase took effect [TIME Thursday] and applies to most routes where it competes against discount carriers, such as [LOC Chicago] to [LOC Dallas] and [LOC Denver] to [LOC San Francisco].

BIO Tagging

A method that lets us turn a segmentation task (finding boundaries of entities) into a classification task

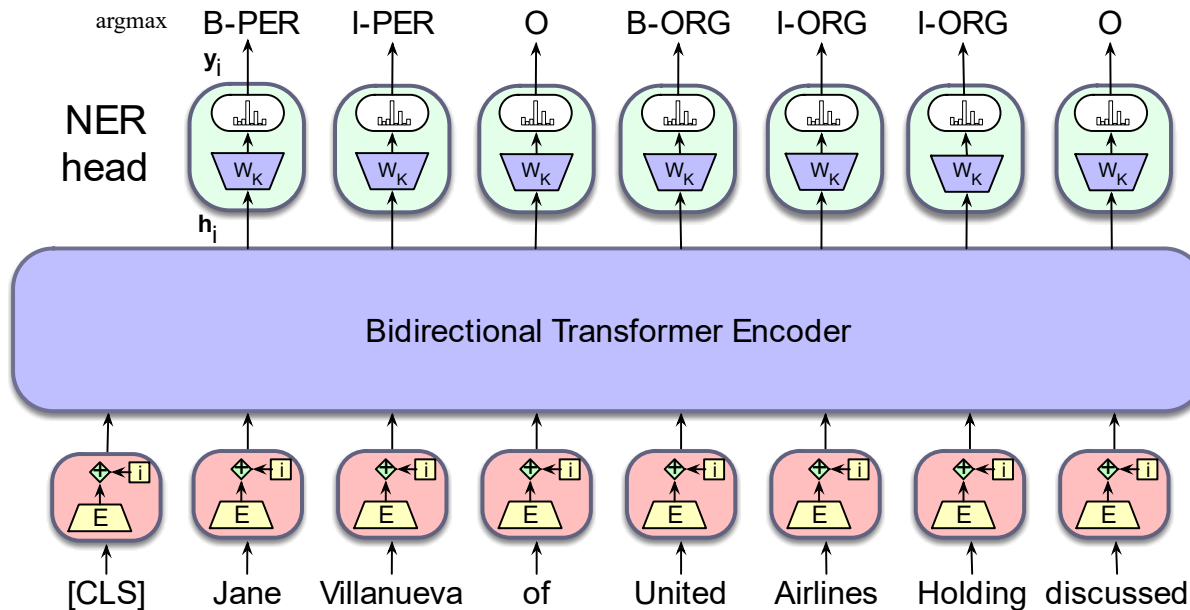
[PER **Jane Villanueva**] of [ORG **United**] , a unit of [ORG **United Airlines Holding**] , said the fare applies to the [LOC **Chicago**] route.

Words	IO Label	BIO Label	BIOES Label
Jane	I-PER	B-PER	B-PER
Villanueva	I-PER	I-PER	E-PER
of	O	O	O
United	I-ORG	B-ORG	B-ORG
Airlines	I-ORG	I-ORG	I-ORG
Holding	I-ORG	I-ORG	E-ORG
discussed	O	O	O
the	O	O	O
Chicago	I-LOC	B-LOC	S-LOC
route	O	O	O
.	O	O	O

Sequence labeling

$$\mathbf{y}_i = \text{softmax}(\mathbf{h}_i^L \mathbf{W}_K)$$

$$\mathbf{t}_i = \text{argmax}_k(\mathbf{y}_i)$$



More details

We need to map between tokens (used by LLM) and words (used in definition of name entities)

We evaluate NER with F1 (precision/recall)

Masked
Language
Models

Fine-Tuning for Classification

Session 11

Unit III

Transformers and Large Language Models

Topic 3.4

Information Retrieval and RAG

Information Retrieval and RAG

- The Information Retrieval Task

Information retrieval

User has an information need

And has some collection of documents

User wants to find a **relevant** document

- a document (or documents)
- in the collection
- that satisfy their need

Web search



Search bar with a magnifying glass icon on the left and three icons on the right: a microphone, a camera, and a button labeled "AI Mode" with a colorful underline.

Google Search

I'm Feeling Lucky

Not just the web

Searching our email

Searching corporate documents

Searching personal medical records

And also, part of LLMs

- retrieval-augmented generation (RAG)

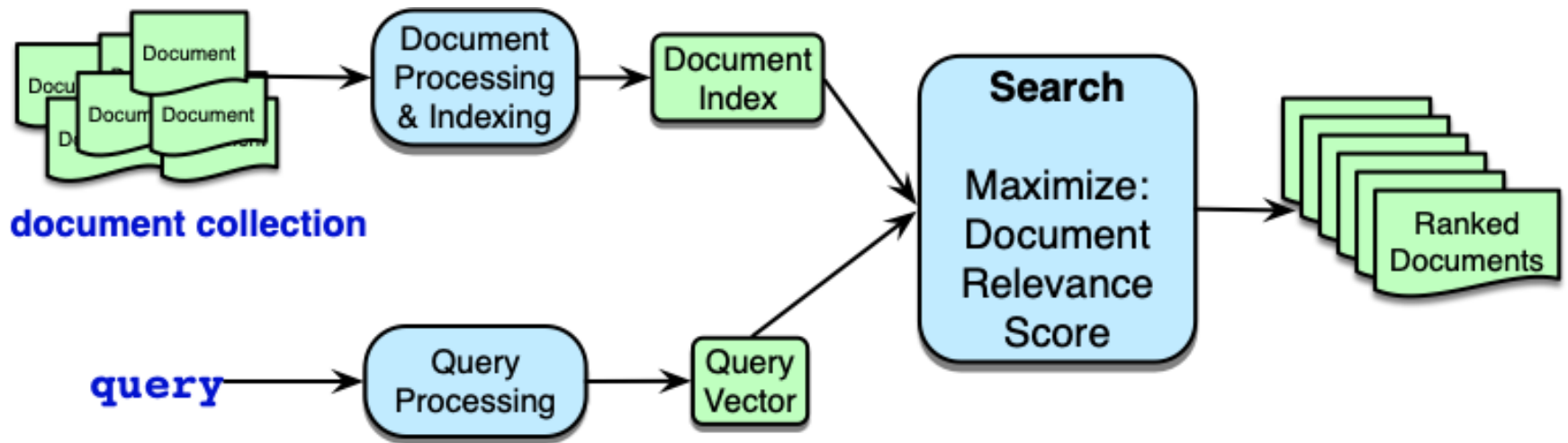
In most cases we do "ranked retrieval"

The retriever returns top-k documents

These are ranked

We can show the user these, or some subset.

Ad-hoc retrieval



Document Relevance Score

Goal is to assign a score to each document for whether it meets the user's information need

Instead, we just approximate this by the textual similarity between the query and the document.

Two architectures

Sparse retrieval

- represent query and doc as vectors of word counts
- weighted by tf-idf, BM25

Dense retrieval

- Use LLM to represent query and doc as embeddings

In both cases, similarity is dot product or cosine between query and document representations

Information Retrieval and RAG

- The Information Retrieval Task

Information Retrieval and RAG

- Sparse retrieval: the vector model of IR

The vector space model of IR

Gerard Salton, 1971

Represent a document as a vector of counts of the words it contains.

Bag-of-words model

I love this movie! It's sweet,
but with satirical humor. The
dialogue is great and the
adventure scenes are fun...
It manages to be whimsical
and romantic while laughing
at the conventions of the
fairy tale genre. I would
recommend it to just about
anyone. I've seen it several
times, and I'm always happy
to see it again whenever I
have a friend who hasn't
seen it yet!

Vector representation of that doc

[1 3 1 1 1 1 1 5 6 1 2 1 4 1 3 1 1 1]

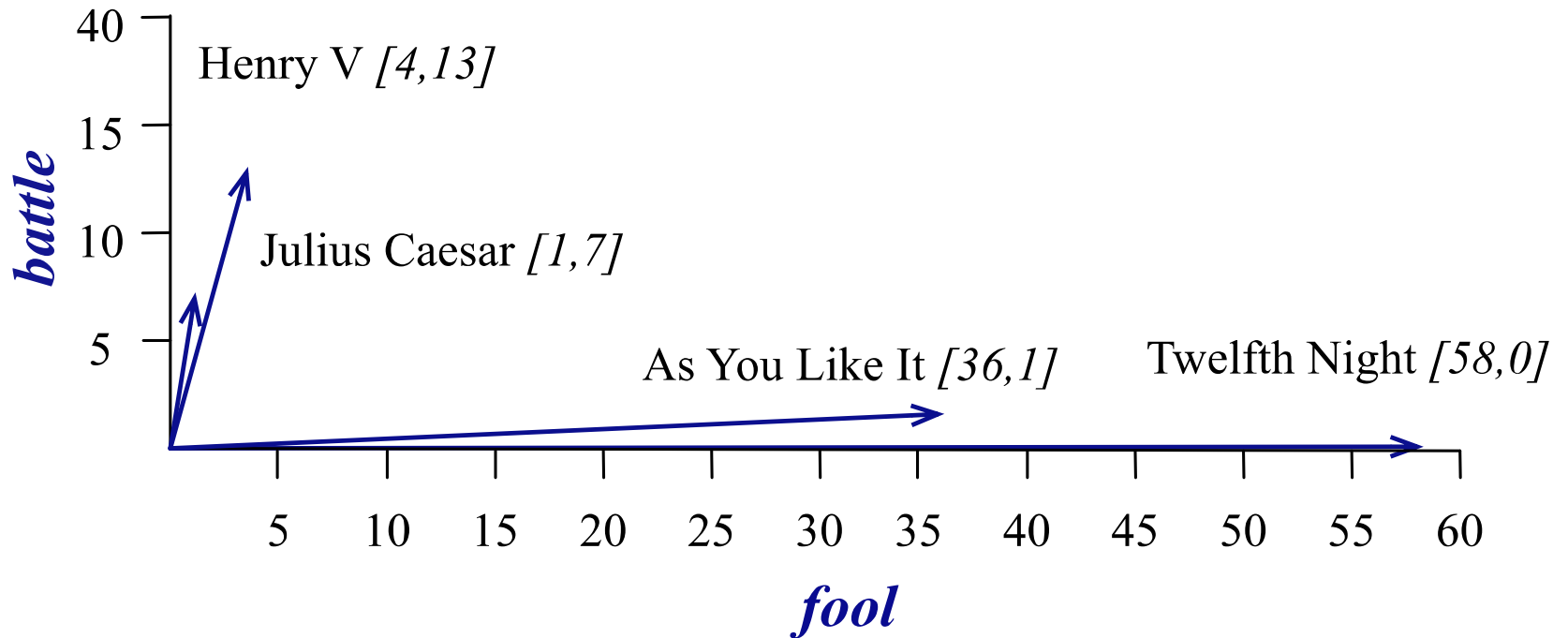
Term-document matrix

Each document is represented by a vector of words

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Visualizing document vectors

The two dimensional space [battle, fool]



Vectors are the basis of information retrieval

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Vectors are similar for the two comedies

But comedies are different than the other two

Comedies have more *fools* and *wit* and fewer *battles*.

Vector representations of queries and documents

Suppose we are looking for a witty fool play:

Query = "fool wit"

	As You Like It	Twelfth Night	Julius Caesar	Henry V	Query
battle	1	0	7	13	0
good	114	80	62	89	0
fool	36	58	1	4	1
wit	20	15	2	3	1

Choose the document that is most similar to the query

Which of d_1 , d_2 , d_3 , d_4 is most similar to q ?

	d_1	d_2	d_3	d_4	q
	As You Like It	Twelfth Night	Julius Caesar	Henry V	Query
battle	1	0	7	13	0
good	114	80	62	89	0
fool	36	58	1	4	1
wit	20	15	2	3	1

Similarity methods are variants of dot product

The dot product is $\mathbf{q} \cdot \mathbf{d}$

$$\text{score}(\mathbf{q}, \mathbf{d}_1) = \mathbf{q} \cdot \mathbf{d}_1 =$$

	d_1	d_2	d_3	d_4	q
	As You Like It	Twelfth Night	Julius Caesar	Henry V	Query
battle	1	0	7	13	0
good	114	80	62	89	0
fool	36	58	1	4	1
wit	20	15	2	3	1

In fact we use cosine

$$\text{score}(q, d) = \cos(q, d) = \frac{q \cdot d}{|q||d|}$$

In fact we use cosine

$$\begin{aligned}\text{score}(q, d) &= \cos(q, d) = \frac{q \cdot d}{\|q\| \|d\|} \\ &= \frac{q}{\|q\|} \cdot \frac{d}{\|d\|}\end{aligned}$$

Information Retrieval and RAG

- Sparse retrieval: the vector model of IR

Information Retrieval and RAG

- TF-IDF

But raw frequency is a bad representation

- The co-occurrence matrices we have seen represent each cell by word frequencies.
- Frequency is clearly useful; if *sugar* appears a lot near *apricot*, that's useful information.
- But overly frequent words like *the*, *it*, or *they* are not very informative about the context
- It's a paradox! How can we balance these two conflicting constraints?

Two common solutions for word weighting

tf-idf: tf-idf value for word t in document d :

$$w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

PMI (Pointwise mutual information)
Words like "the" or "a" have very low idf

$$-\text{PMI}(w_1, w_2) = \log \frac{p(w_1, w_2)}{p(w_1)p(w_2)}$$

See if words like "good" appear more often with "great" than we would expect by chance

Term frequency (tf) in the tf-idf algorithm

We could imagine using raw count:

$$\text{tf}_{t,d} = \text{count}(t,d)$$

But instead of using raw count, we usually squash a bit:

$$\text{tf}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t,d) & \text{if } \text{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

Document frequency (df)

df_t is the number of documents t occurs in.

(note this is not collection frequency: total count across all documents)

"*Romeo*" is very distinctive for one Shakespeare play:

	Collection Frequency	Document Frequency
Romeo	113	1
action	113	31

Inverse document frequency (idf)

$$\text{idf}_t = \log_{10} \left(\frac{N}{\text{df}_t} \right)$$

N is the total number of documents
in the collection

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

What is a document?

Could be a play or a Wikipedia article

But for the purposes of tf-idf, documents can be **anything**; we often call each paragraph a document!

Final tf-idf weighted value for a word

Raw counts:

$$w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.246	0	0.454	0.520
good	0	0	0	0
fool	0.030	0.033	0.0012	0.0019
wit	0.085	0.081	0.048	0.054

Information Retrieval and RAG

- TF-IDF

Information Retrieval and RAG

- TF-IDF: a worked example

Cosine

$$\text{score}(q, d) = \cos(q, d) = \frac{q \cdot d}{|q||d|} \\ = \frac{q}{|q|} \cdot \frac{d}{|d|}$$

TF-IDF weighted cosine

$$\text{score}(q, d) = \cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q}}{|\mathbf{q}|} \cdot \frac{\mathbf{d}}{|\mathbf{d}|}$$

$$= \sum_{t \in \mathbf{q}} \frac{\text{tf-idf}(t, q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i, q)}} \cdot \frac{\text{tf-idf}(t, d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i, d)}}$$

TF-IDF weighted cosine

$$\text{tf}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t,d) & \text{if } \text{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\text{idf}_t = \log_{10} \left(\frac{N}{\text{df}_t} \right)$$

$$\text{score}(q,d) = \sum_{t \in q} \frac{\text{tf-idf}(t,q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf-idf}(t,d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i,d)}}$$

TF-IDF nano-example

Query:	sweet love
Doc 1:	Sweet sweet nurse! Love?
Doc 2:	Sweet sorrow
Doc 3:	How sweet is love?
Doc 4:	Nurse!

TF-IDF nano-example

Query: sweet love

$$|q| = \sqrt{\quad}$$

Word	Count	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized
sweet	1					
nurse	0					
love	1					
how	0					
sorrow	0					
is	0					

TF-IDF nano-example

Query: sweet love

$$|q| = \sqrt{.125^2 + .301^2} = .325$$

Word	Count	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized
sweet	1	1	3	0.125	0.125	0.383
nurse	0					
love	1	1	2	0.301	0.301	0.924
how	0					
sorrow	0					
is	0					

TF-IDF nano-example

Doc 2: Sweet sorrow

$$|q| = \sqrt{\quad}$$

Word	Count	tf $1 + \log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized	$\times q$
sweet	1						
nurse	0						
love	0						
how	0						
sorrow	1						
is	0						

TF-IDF nano-example

Doc 2: Sweet sorrow

$$|q| = \sqrt{.125^2 + .602^2} = .615$$

Word	Cnt	tf $1 + \log_{10}(c)$	df	idf $\log_{10}N/df$	tf-idf tf x idf	normalized	× q
sweet	1	1	3	0.125	.125	.203	0.0779
nurse	0						
love	0						
how	0						
sorrow	1	1	1	.602	0.602	.979	0
is	0						

TF-IDF nano-example

Query: sweet love

· **Doc 2:** Sweet sorrow

= 0.0779

Word	Cnt	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized	× q
sweet	1	1	3	0.125	.125	.203	0.0779
nurse	0						
love	0						
how	0						
sorrow	1	1	1	.602	0.602	.979	0
is	0						

Final cosine

(details in
chapter)

Query:	sweet love
Doc 1:	Sweet sweet nurse! Love?
Doc 2:	Sweet sorrow
Doc 3:	How sweet is love?
Doc 4:	Nurse!

$\text{score}(q, d1) = 0.747$

$\text{score}(q, d2) = 0.0779$

- d1 has both terms, including 2 instances of *sweet*
- d2 is missing one of the terms

Information Retrieval and RAG

- TF-IDF: a worked example

Information Retrieval and RAG

- Efficiency: The Inverted Index

Goal: rank documents in **D** by their TF-IDF-weighted cosines with query **q**

Do we have to consider all documents in D?

No! **We can ignore all documents that don't have any query words!**

They will have a cosine of 0!

How do we efficiently find all documents that contain a query term q_i ?

An index!

Which for historical reasons we call an inverted index!

Inverted Index

Doc 1:	Sweet sweet nurse! Love?
Doc 2:	Sweet sorrow
Doc 3:	How sweet is love?
Doc 4:	Nurse!

Two parts

Dictionary: Postings:

how	→ 3
is	→ 3
love	→ 1 → 3
nurse	→ 1 → 4
sorrow	→ 2
sweet	→ 1 → 2 → 3

Inverted Index Creation

1. Sort by term and document

2. Create linked postings list

Doc 1:	Sweet sweet nurse! Love?
Doc 2:	Sweet sorrow
Doc 3:	How sweet is love?
Doc 4:	Nurse!

→

Term	Doc#
sweet	1
sweet	1
nurse	1
love	1
sweet	2
sorrow	2
how	3
sweet	3
is	3
love	3
nurse	4

→

Term	Doc#
how	3
is	3
love	1
love	3
nurse	1
nurse	4
sorrow	2
sweet	2
sweet	1
sweet	1
sweet	3

→

Dict: Postings:

how → 3
is → 3
love → 1 → 3
nurse → 1 → 4
sorrow → 2
sweet → 1 → 2 → 3

Inver

Dict Postings

how	→ 3
is	→ 3
love	→ 1 → 3
nurse	→ 1 → 4
sorrow	→ 2
sweet	→ 1 → 2 → 3

So far this just tells us which documents to grab

Next we need enough information to compute tf-idf:

For each term t in vocabulary:

- The df_t (document frequency) of word t

For each term t in each document d :

- The term frequency or $\text{count}(t,d)$ of word t in doc d

Doc 1:	Sweet sweet nurse! Love?
Doc 2:	Sweet sorrow
Doc 3:	How sweet is love?
Doc 4:	Nurse!

		Doc#	tf
how {1}	→	3	[1]
is {1}	→	3	[1]
love {2}	→	1 [1] → 3 [1]	
nurse {2}	→	1 [1] → 4 [1]	
sorry {1}	→	2 [1]	
sweet {3}	→	1 [2] → 2 [1] → 3 [1]	

df



tf

Information Retrieval and RAG

- Efficiency: The Inverted Index

Information Retrieval and RAG

- Evaluation of IR

Precision and Recall

We saw these already for classification

		<i>gold standard labels</i>		
		gold positive	gold negative	
<i>system output labels</i>	system positive	true positive	false positive	precision = $\frac{tp}{tp+fp}$
	system negative	false negative	true negative	
		recall = $\frac{tp}{tp+fn}$		accuracy = $\frac{tp+tn}{tp+fp+tn+fn}$

Precision: % of selected items that are correct

Recall: % of correct items that are selected

Precision and Recall for IR

User makes an information request

Every document in collection is either:

- **Relevant** to the user
- **Not relevant** to the user

The system retrieves a ranked set of documents

Precision for IR

Precision = % of **retrieved** documents that are **relevant**

System retrieves two kinds of documents

relevant documents

irrelevant documents

$$\text{Precision} = \frac{|\text{relevant retrieved docs}|}{|\text{relevant retrieved docs}| + |\text{irrelevant retrieved docs}|}$$

Recall for IR

Recall = % of **relevant** documents that are **retrieved**

$$\text{Recall} = \frac{|\text{retrieved relevant documents}|}{|\text{all relevant document}|}$$

Precision and Recall aren't enough

This is **ranked** retrieval

- Given two ranked retrieval systems
- We want a metric that prefers the one that **ranks relevant documents higher**

We need to adapt precision and recall!

- to be sensitive to **where in the ranking** the relevant document occur

Rank-specific precision and r

Rank	Judgment	Precision _{R_{cutoff}}	Recall _{R_{cutoff}}
1	R	1.0	.11

Example:

- 25 documents
- 9 are relevant
- If we return all 25
P=.36, R= 1.0
- Suppose we just return one (and it is relevant)?

Rank-specific precision and recall

Recall is non-decreasing

- Relevant docs increase recall
- Non-relevant docs don't

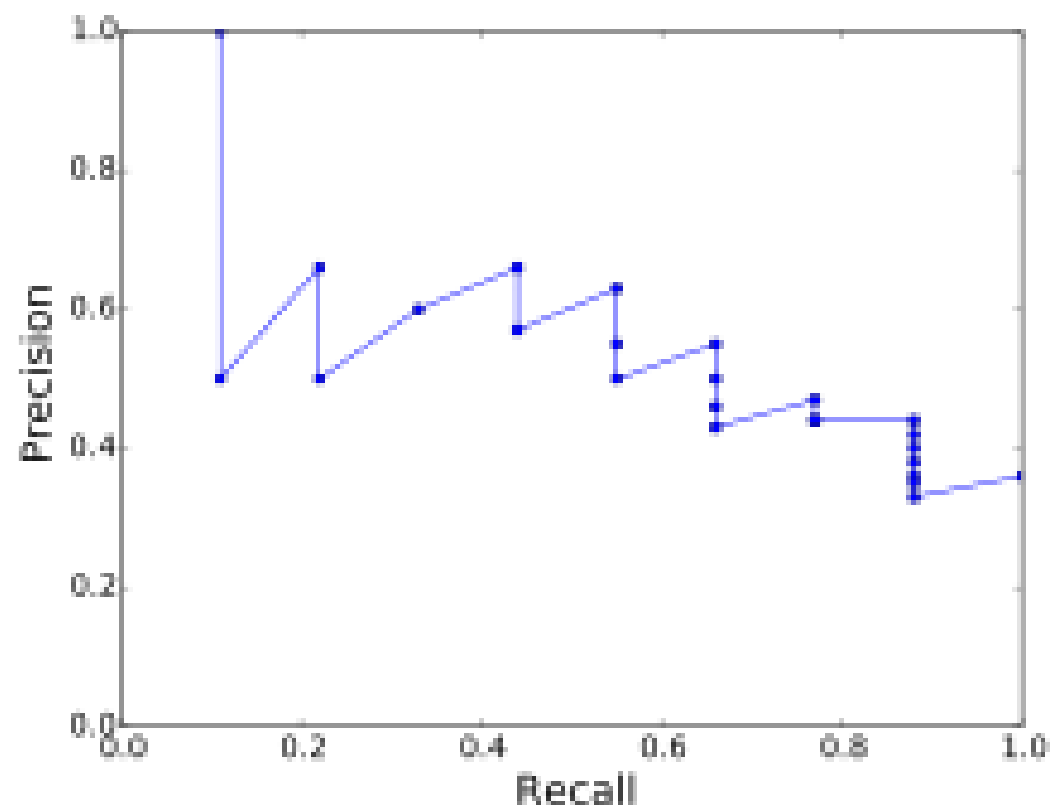
Precision jumps up and down

- increasing for relevant docs
- decreasing otherwise.

Rank	Judgment	Precision _{Rank}	Recall _{Rank}
1	R	1.0	.11
2	N	.50	.11
3	R	.66	.22
4	N	.50	.22
5	R	.60	.33
6	R	.66	.44
7	N	.57	.44
8	R	.63	.55
9	N	.55	.55
10	N	.50	.55
11	R	.55	.66
12	N	.50	.66
13	N	.46	.66
14	N	.43	.66
15	R	.47	.77
16	N	.44	.77
17	N	.44	.77
18	R	.44	.88
19	N	.42	.88
20	N	.40	.88
21	N	.38	.88
22	N	.36	.88
23	N	.35	.88
24	N	.33	.88
25	R	.36	1.0

The precision-recall curve (for one query)

Rank	Judgment	Precision _{Rank}	Recall _{Rank}
1	R	1.0	.11
2	N	.50	.11
3	R	.66	.22
4	N	.50	.22
5	R	.60	.33
6	R	.66	.44
7	N	.57	.44
8	R	.63	.55
9	N	.55	.55
10	N	.50	.55
11	R	.55	.66
12	N	.50	.66
13	N	.46	.66
14	N	.43	.66
15	R	.47	.77
16	N	.44	.77
17	N	.44	.77
18	R	.44	.88
19	N	.42	.88
20	N	.40	.88
21	N	.38	.88
22	N	.36	.88
23	N	.35	.88
24	N	.33	.88



Need a metric that aggregates over many queries

Two common approaches

- Mean Average Precision
- Interpolated Precision

Mean Average Precision

Descend through ranked items

Note precision only if item is **relevant**

- e.g. ranks 1, 3, 5, 6 but not 2 or 4:

$\text{Precision}_r(d)$ "ranked precision"

precision at the rank doc d was found.

Rank	Judgment	Precision_r	Recall_r
1	R	1.0	.11
2	N	.50	.11
3	R	.66	.22
4	N	.50	.22
5	R	.60	.33
6	R	.66	.44
7	N	.57	.44

Average Precision

Descend through ranked items

Note precision only if item is **relevant**

Take the average of the ranked-precisions

$$AP = \frac{1}{|R_r|} \sum_{d \in R_r} \text{Precision}_r(d)$$

- R_r is the set of relevant documents at or above r
- $\text{Precision}_r(d)$ is precision measured at the rank at which document d was found.

Mean Average Precision

For a set of Q queries

Mean Average Precision (MAP):

$$\text{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP}(q)$$

Information Retrieval and RAG

- Evaluation of IR

Information Retrieval and RAG

- Dense Retrieval

Problem with classic IR

The **vocabulary mismatch problem**

- tf-idf or BM25 cosine similarities only work if there is **exact word overlap** between query and doc!
- But query-writer can't know the exact words the doc might include!

Dense retrieval

Instead of representing query and documents with count vectors
Represent both with embeddings!

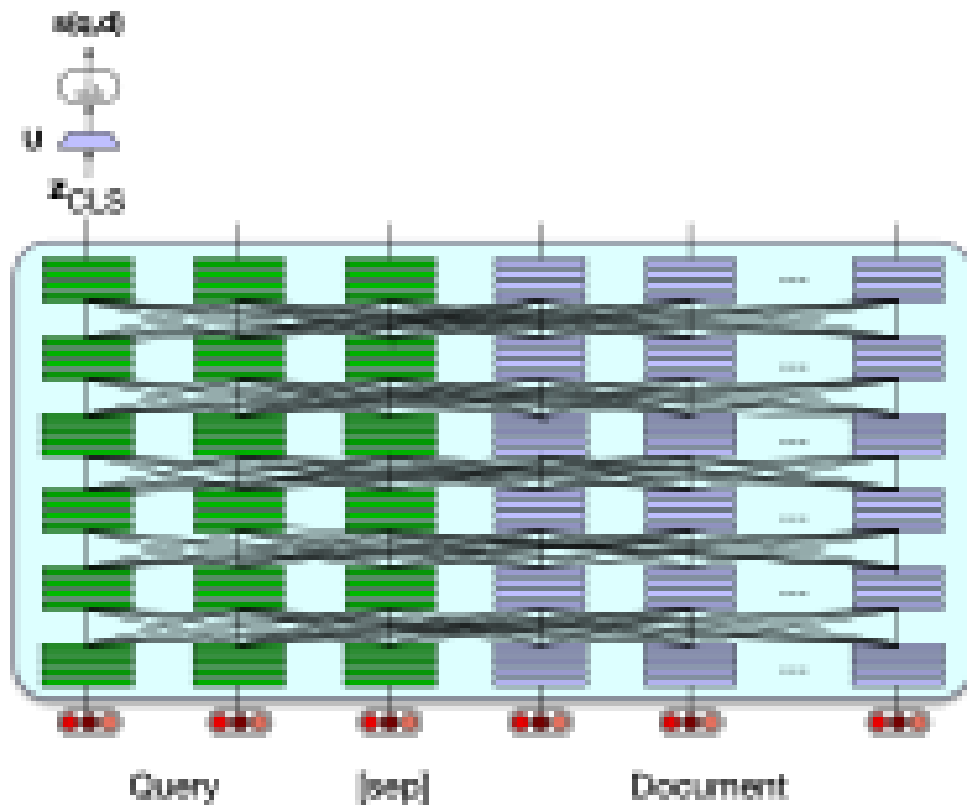
Hypothetical version of dense retrieval: static embeddings

Replace tf-idf vectors with, e.g., word2vec

- Query: the mean of the embeddings of each query word
- Doc: the mean of the doc word embeddings
- Now just compute query-doc cosine as normal.

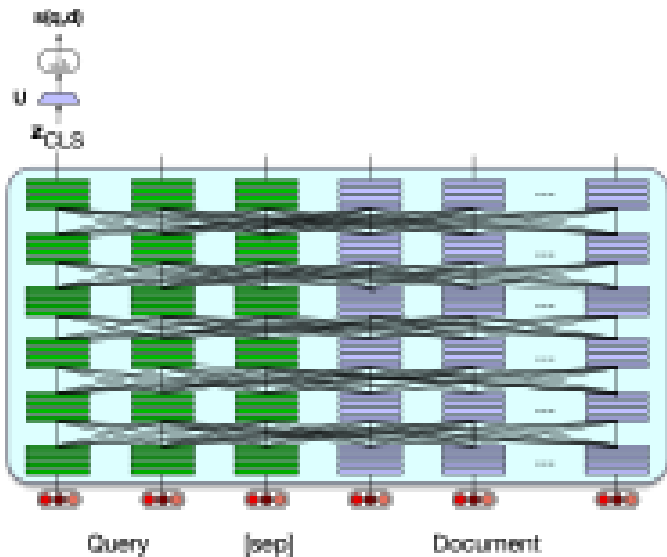
We don't do this because **contextual embeddings** work much better!

Dense retrieval #1: Single encoder



$$\mathbf{z} = \text{BERT}(q; [\text{SEP}]; d) [\text{CLS}]$$
$$\text{score}(q, d) = \text{softmax}(\mathbf{U}(\mathbf{z}))$$

Dense retrieval #1: Single encoder

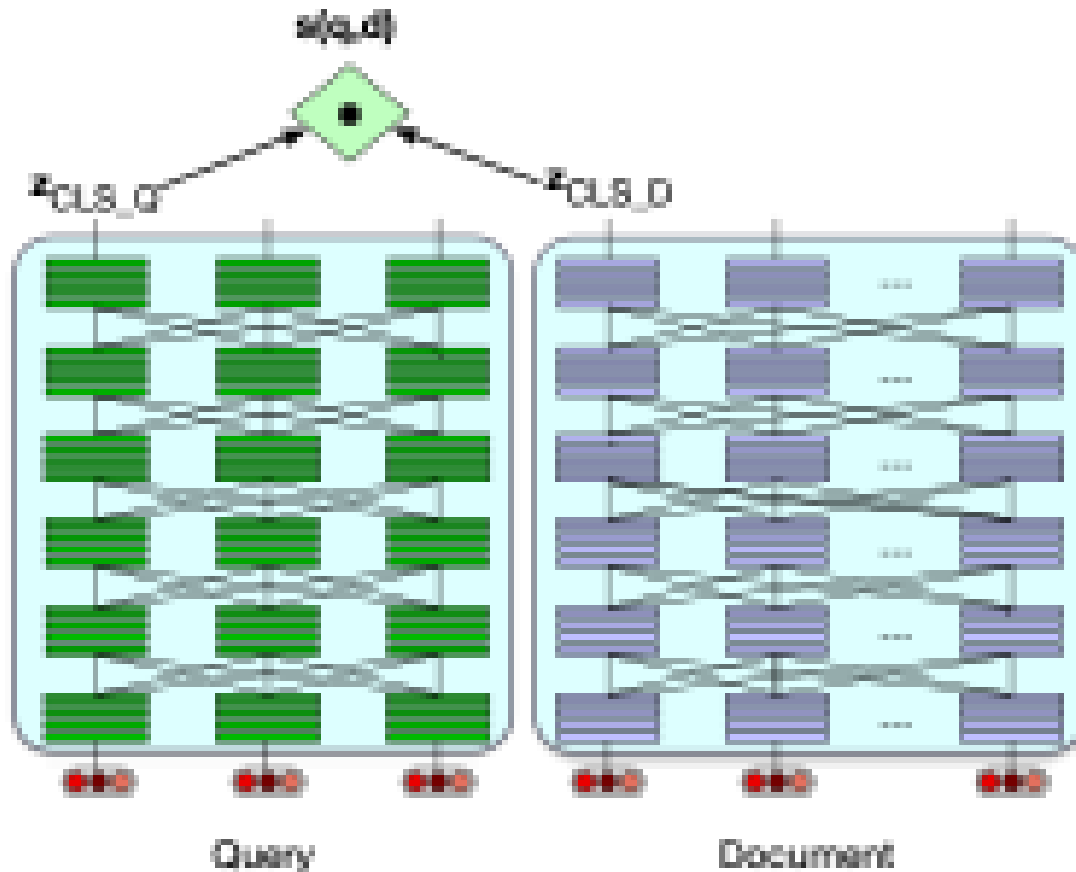


System is run on passages (say 100 tokens) instead of whole documents

Training:

- BERT and linear layer U can then fine-tuned for relevance
- Creating a tuning dataset of relevant and non-relevant passages.

Dense retrieval #2 (biencoder)



$$\mathbf{z}_q = \text{BERT}_Q(q) [\text{CLS}]$$

$$\mathbf{z}_d = \text{BERT}_D(d) [\text{CLS}]$$

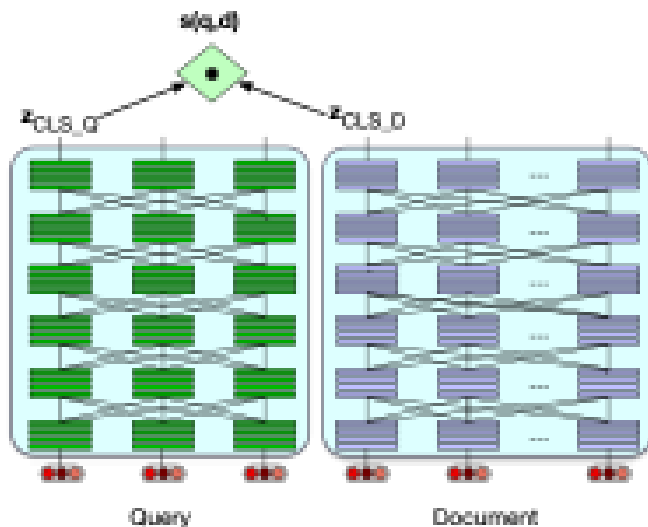
$$\text{score}(q, d) = \mathbf{z}_q \cdot \mathbf{z}_d$$

Dense retrieval #2 (**biencoder**)

Encode doc vectors in advance.

Encode query when it arrives

- Score is dot product between query vector and precomputed doc vector
- Cheaper but less accurate



In-between dense retrieval methods

Use cheap methods (like BM25) as first pass relevance ranking for each document,

Then just rerank the top N ranked docs,

- Using expensive methods like the full BERT scoring

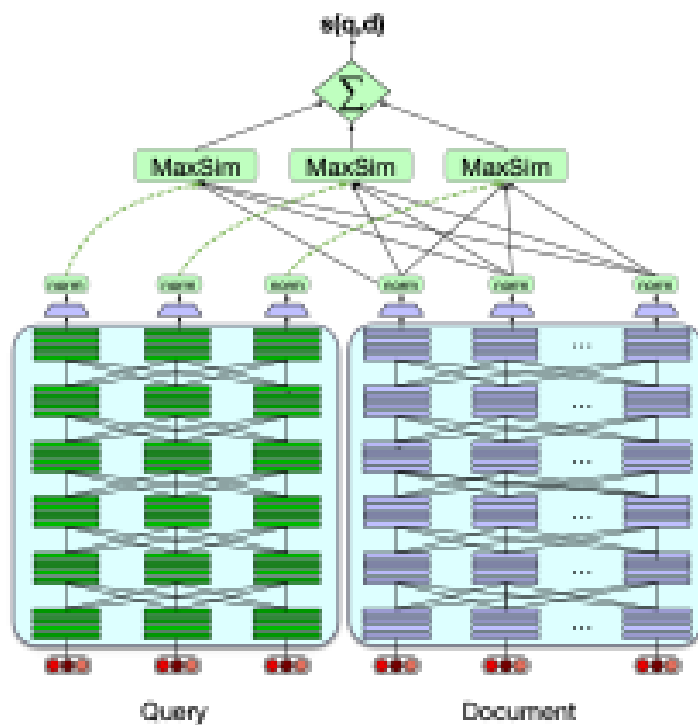
ColBERT

Precompute document but store each word vector

Then do maxsim between document and query words

Khattab and Zaharia (2020), Khattab et al (2021)

CoBERT



$$\text{score}(q, d) = \sum_{i=1}^N \max_{j=1}^M \mathbf{E}_{q_i} \cdot \mathbf{E}_{d_j}$$

Training for dense retrieval

ColBERT and other models need to be trained

- To fine-tune the BERT encoders and train the linear layers (and the special [Q] and [D] embeddings)
- On datasets of triples $\langle q, d+, d- \rangle$
- Some datasets like MS MARCO Ranking have positive examples

Efficiency in dense retrieval

We must rank **every document** for its similarity to the query!

Efficiency for sparse word-count vectors: inverted index

Efficiency for dense retrieval:

- **nearest neighbor search:** finding the set of dense document vectors that have the highest dot product with a dense query vector.
- Approximate nearest neighbor algorithm **Faiss** (Johnson et al., 2017).
 - Approximates the doc vector by a smaller quantized vector

Information Retrieval and RAG

- Dense Retrieval

Information Retrieval and RAG

- Retrieval-Augmented Generation

IR plays a central role in modern LLMs

How to answer factual questions like

- Where is the Louvre Museum located?
- How to get a script I in latex?
- Where does the energy in a nuclear explosion come from?

Just prompt an LLM!

◆ AI Overview

The main Louvre Museum is located in **Paris, France**, at the **Musée du Louvre**, 75001 Paris, France. It is situated on the Right Bank of the Seine River in the city's 1st arrondissement, housed within the historic Louvre Palace. 

- **Address:** Rue de Rivoli, 75001 Paris, France.

LLMs seem to store facts in the connections in their feedforward layers!

But there are issues!

LLMs Hallucinate

Hallucination: a response that is not faithful to the facts of the world.

In the legal domain LLMs were shown to hallucinate up to 88% of the time!

Dahl et al. (2024)

Can't use Proprietary Data

People need to ask questions about:

- personal email.
- healthcare applications to medical records.
- internal corporate documents
- legal documents discovery

Can't Handle Dynamic Data

LLMs can't answer questions about rapidly changing information

Things that happened last week

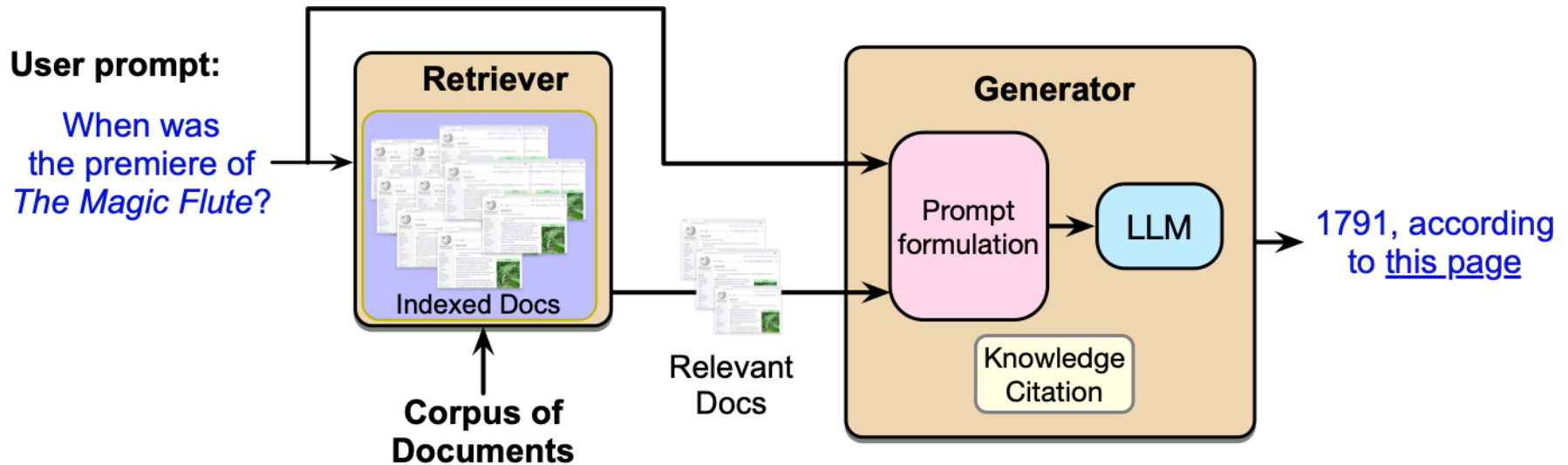
In general, data shifts over time

Solution: RAG

Retrieval-Augmented Generation

1. Use IR to **retrieve** documents from some collection
2. Then use LLM to **generate** an answer conditioned on the documents

Retrieval Augmented Generation (RAG)



Basic RAG

Given a document collection D and a user query q

- Call a retriever to return top k passages
- Create a prompt that includes q and the passages
- Call an LLM with the prompt

Schematic of a RAG Prompt

retrieved passage 1

retrieved passage 2

...

retrieved passage k

Based on these texts, answer this question: What year was the premiere of *The Magic Flute*?

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i | \text{R}(q) ; \text{Answer the following question...} ; q ; x_{<i})$$

Extensions: Agent-based RAG

Instead of running RAG automatically on every user turn

Have a retrieval agents

System decides when to call it and for which document collection

Extensions: Training

Instruction-tune an LLM on a dataset of questions with retrieved passages and correct answers

Test-time compute: prompt an LLM to answer the question and simultaneously to generate reflections on which passages were useful

Extensions: Knowledge Citations

Q: Which films have Gong Li as a member of their cast?

A: The Story of Qiu Ju [1], Farewell My Concubine [2], The Monkey King 2 [3], Mulan [3], Saturday Fiction [3] ...

“Write an answer for the given question using only the provided search results (some of which might be irrelevant) and cite them properly... Always cite for any factual claim”.

Information Retrieval and RAG

- Retrieval-Augmented Generation

Information Retrieval and RAG

- Question Answering
datasets and evals

Two kinds of question answering datasets

Natural information-seeking questions

- Someone actually wanted to know the answer to this

Probing (testing) questions

- Exam-type questions for LLM evaluation

Natural Questions (Kwiatkowski et al., 2019),

anonymized English **queries** to the Google search engine and short and long **answers** (hand-created from Wikipedia)

“When are hops added to the brewing process?”

short answer: *the boiling process*

long answer: paragraph from the Wikipedia page on *Brewing*

MS MARCO (Microsoft Machine Reading Comprehension) collection of datasets,

1 million real anonymized English questions from Microsoft Bing query logs

human generated answer

9 million passages (Bajaj et al., 2016)

Probing dataset: MMLU

15908 knowledge and reasoning questions in 57 areas including medicine, mathematics, computer science, law, etc..

Sourced from exams for humans like GRE, AP

College Computer Science

Any set of Boolean operators that is sufficient to represent all Boolean expressions is said to be complete. Which of the following is NOT complete?

- (A) AND, NOT
- (B) NOT, OR
- (C) AND, OR
- (D) NAND

Information Retrieval and RAG

- Question Answering datasets and evals